Amortized Analysis

Bin Wang

School of Software
Tsinghua University

November 4, 2009

Outline

1. Dynamic tables
2. Aggregate analysis
3. The accounting method
4. The potential method

How large should a hash table or an array be?

Goal

Make the table as small as possible, but large enough so that it won't overflow.

Problem

What if we don't know the proper size in advance?

How large should a hash table or an array be?

Goal

Make the table as small as possible, but large enough so that it won't overflow.

Problem

What if we don't know the proper size in advance?

How large should a hash table or an array be?

Solution

Dynamic tables.

Whenever the table overflows, “grow” it by allocating (via **malloc** or **new**) a new, larger table. Move all items from the old table into the new one, and free the storage for the old table.

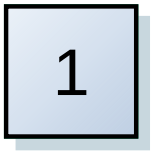
How large should a hash table or an array be?

Solution

Dynamic tables.
Whenever the table overflows, “grow” it by allocating (via **malloc** or **new**) a new, larger table. Move all items from the old table into the new one, and free the storage for the old table.

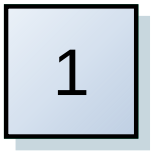
Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



Example of a dynamic table

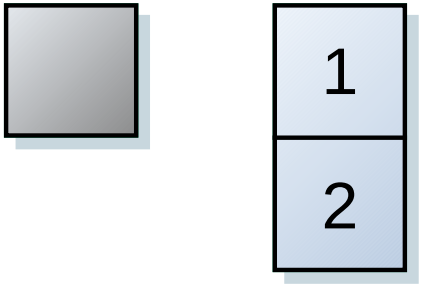
1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



overflow

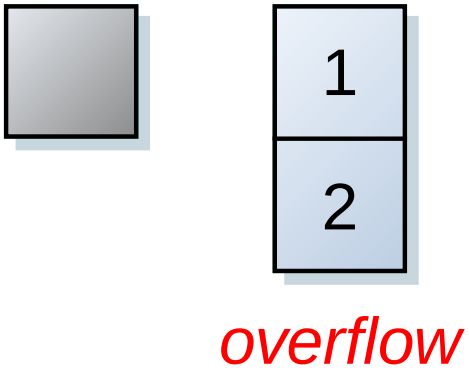
Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



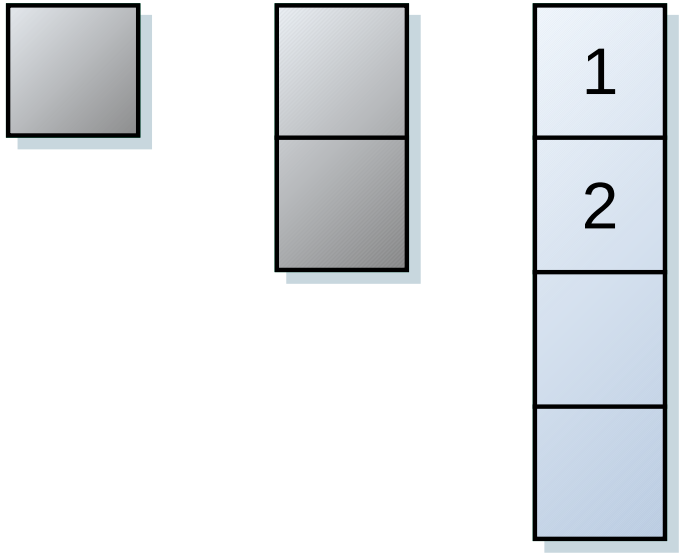
Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



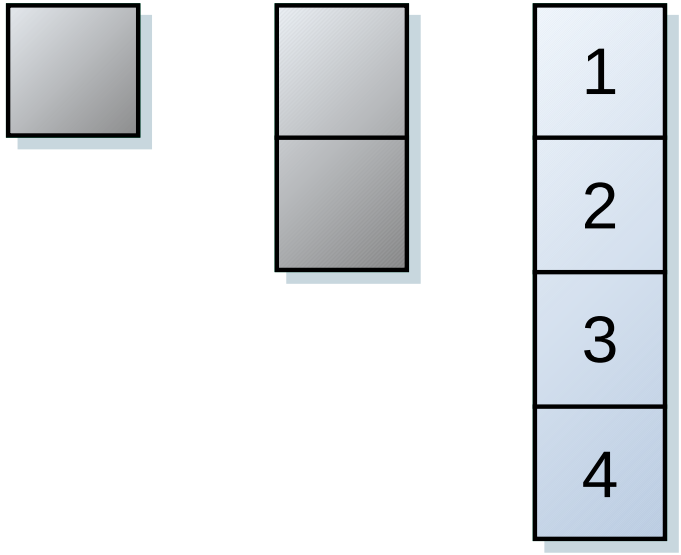
Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



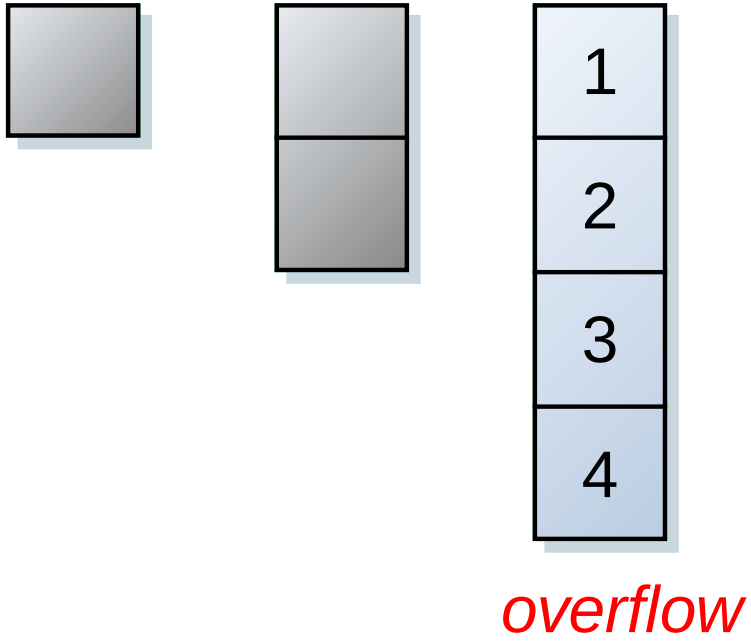
Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



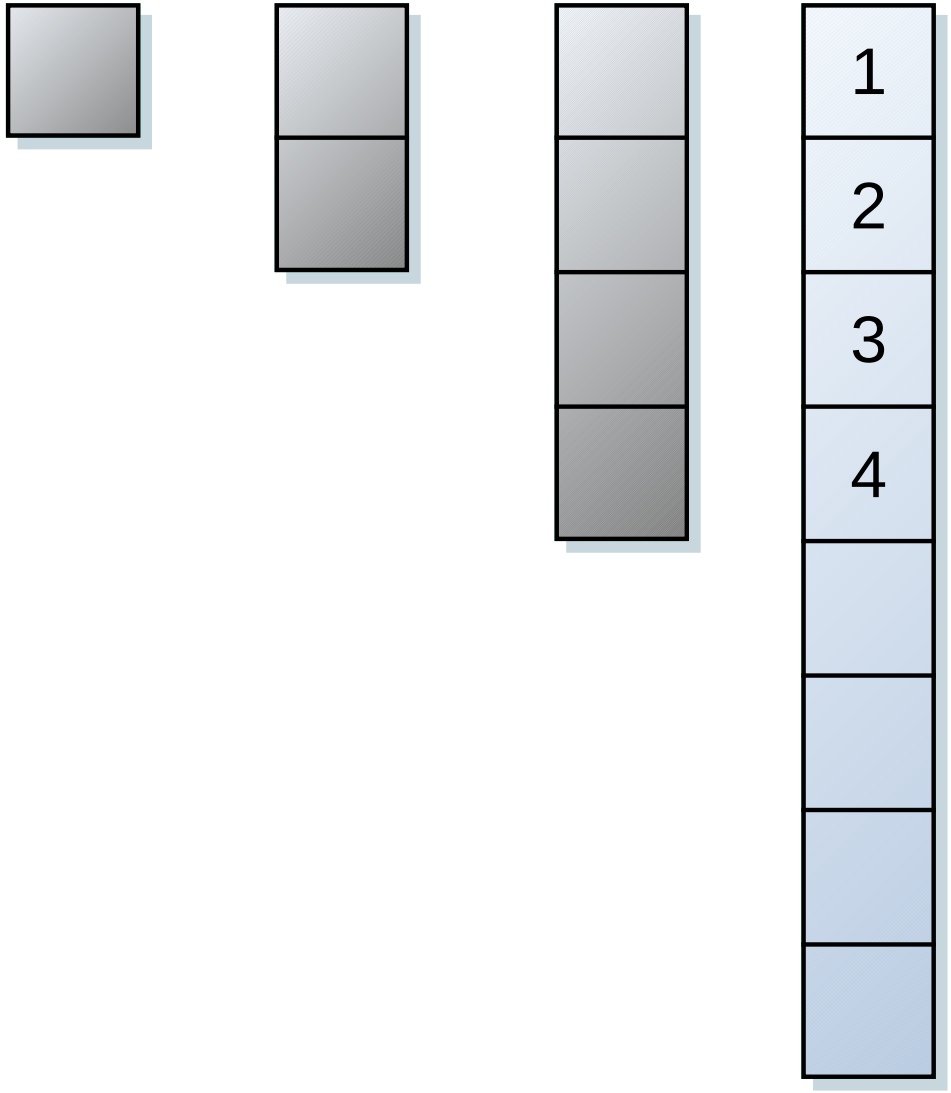
Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT



Example of a dynamic table

1. INSERT
2. INSERT
3. INSERT
4. INSERT
5. INSERT
6. INSERT
7. INSERT

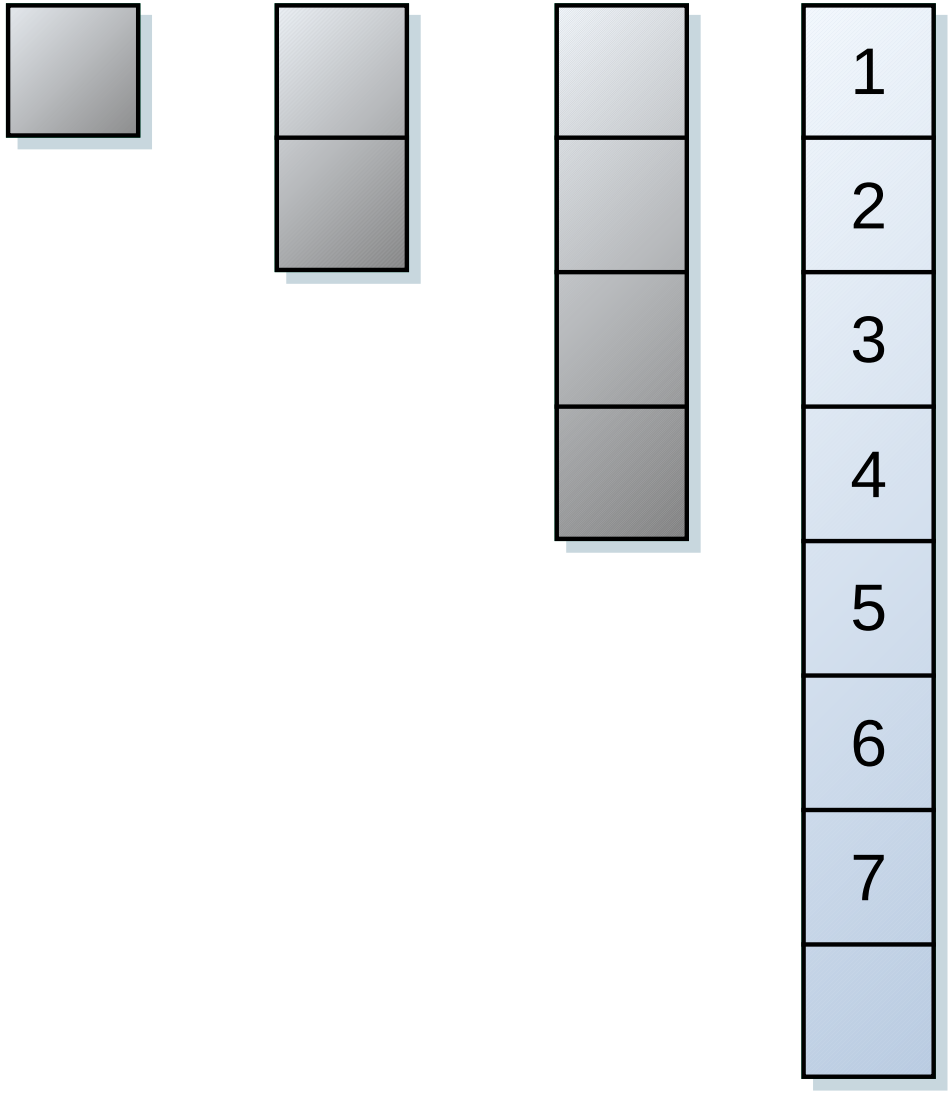


Table-Insert

TABLE-INSERT(T, x)

```
if size[T] = 0
    then allocate table[T] with 1 slot
         size[T] ← 1
if num[T] = size[T]
    then allocate new-table with 2 · size[T] slots
         insert all items in table[T] into new−table
         free table[T]
         table[T] ← new−table
         size[T] ← 2 · size[T]
insert x into table[T]
num[T] ← num[T] + 1
```

Worse-case analysis

A cursory analysis

Consider a sequence of n insertions. The worst-case time to execute one insertion is $\Theta(n)$. Therefore, the worst-case time for n insertions is $n \cdot \Theta(n) = \Theta(n^2)$.

WRONG!

In fact, the worst-case cost for n insertions is only $\Theta(n) \ll \Theta(n^2)$

Worse-case analysis

A cursory analysis

Consider a sequence of n insertions. The worst-case time to execute one insertion is $\Theta(n)$. Therefore, the worst-case time for n insertions is $n \cdot \Theta(n) = \Theta(n^2)$.

WRONG!

In fact, the worst-case cost for n insertions is only $\Theta(n) \ll \Theta(n^2)$

Worse-case analysis

A tighter analysis

Let c_i be the cost of the ith insertion.

$$c_i = \begin{cases} i & \text{if } i - 1 = 2^m, m = 0, 1, 2, \ldots \\ 1 & \text{otherwise} \end{cases}$$

i	1	2	3	4	5	6	7	8	9
$size_i$									
c_i									

Worse-case analysis

A tighter analysis

Let c_i be the cost of the ith insertion.

$$c_i = \begin{cases} i & \text{if } i - 1 = 2^m, m = 0, 1, 2, \ldots \\ 1 & \text{otherwise} \end{cases}$$

i	1	2	3	4	5	6	7	8	9
$size_i$									
c_i									

Worse-case analysis

A tighter analysis

Let c_i be the cost of the ith insertion.

$$c_i = \begin{cases} i & \text{if } i - 1 = 2^m, m = 0, 1, 2, \ldots \\ 1 & \text{otherwise} \end{cases}$$

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i									

Worse-case analysis

A tighter analysis

Let c_i be the cost of the ith insertion.

$$c_i = \begin{cases} i & \text{if } i - 1 = 2^m, m = 0, 1, 2, \ldots \\ 1 & \text{otherwise} \end{cases}$$

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	1	2	3	1	5	1	1	1	9

Worse-case analysis

A tighter analysis

Let c_i be the cost of the ith insertion.

$$c_i = \begin{cases} i & \text{if } i - 1 = 2^m, m = 0, 1, 2, \ldots \\ 1 & \text{otherwise} \end{cases}$$

i	1	2	3	4	5	6	7	8	9
$size_i$	1	2	4	4	8	8	8	8	16
c_i	1	1	1	1	1	1	1	1	1
		1	2		4				8

Worse-case analysis

A tighter analysis

$$\text{Cost of } n \text{ insertions} = \sum_{i=1}^{n} c_i$$

Worse-case analysis

A tighter analysis

$$\text{Cost of } n \text{ insertions} = \sum_{i=1}^{n} c_i$$

$$\leq n + \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j$$

Worse-case analysis

A tighter analysis

$$\begin{aligned}\text{Cost of } n \text{ insertions} &= \sum_{i=1}^{n} c_i \\ &\leq n + \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j \\ &< 3n\end{aligned}$$

Worse-case analysis

A tighter analysis

$$
\begin{aligned}
\text{Cost of } n \text{ insertions} &= \sum_{i=1}^{n} c_i \\
&\leq n + \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j \\
&< 3n \\
&= \Theta(n)
\end{aligned}
$$

Worse-case analysis

A tighter analysis

$$\begin{aligned}\text{Cost of } n \text{ insertions} &= \sum_{i=1}^{n} c_i \\ &\le n + \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j \\ &< 3n \\ &= \Theta(n)\end{aligned}$$

Thus, the average cost of each dynamic-table operation is $\Theta(n)/n = \Theta(1)$.

Amortized analysis

Definition

An ***amortized analysis*** is any strategy for analyzing a sequence of operations to show that the ***average*** cost per operation is small, even though a single operation within the sequence might be expensive.

Amortized analysis

An amortized analysis guarantees the **average** performance of each operation in the **worst case**.

- the **aggregate** method,
- the **accounting** method,
- the **potential** method.

Amortized analysis

An amortized analysis guarantees the **average** performance of each operation in the **worst case**.

- the **aggregate** method,
- the **accounting** method,
- the **potential** method.

Aggregate analysis

Definition

- In ***aggregate analysis***, we show that for all n, a sequence of n operations takes worst-case time $T(n)$ in total.
- In the worst case, the average cost, or **amortized cost**, per operation is therefore $T(n)/n$.
- This amortized cost applies to each operation, even when there are ***several types*** of operations in the sequence.

Incrementing a binary counter

Definition

- Consider the problem of incrementing a k-bit binary counter that counts upward from 0.
- We use an array $A[0..k-1]$ of bits as the counter.

Incrementing a binary counter

```
INCREMENT(A)
i ← 0
while i < length[A] and A[i] = 1
        do A[i] ← 0
           i ← i + 1
if i < length[A]
    then A[i] ← 1
```

Incrementing a binary counter

A cursory analysis

- A single execution of INCREMENT take time $\Theta(k)$ in the worst case, in which array A contains all 1's.
- Thus, a sequence of n INCREMENT operations on an initially zero counter takes time $O(nk)$ in the worst case.

Incrementing a binary counter

value	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	cost
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	1	
2	0	0	0	0	0	1	0	
3	0	0	0	0	0	1	1	
4	0	0	0	0	1	0	0	
5	0	0	0	0	1	0	1	
6	0	0	0	0	1	1	0	
7	0	0	0	0	1	1	1	
8	0	0	0	1	0	0	0	
9	0	0	0	1	0	0	1	
10	0	0	0	1	0	1	0	

Incrementing a binary counter

value	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	cost
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	1	1
2	0	0	0	0	0	1	0	3
3	0	0	0	0	0	1	1	4
4	0	0	0	0	1	0	0	7
5	0	0	0	0	1	0	1	8
6	0	0	0	0	1	1	0	
7	0	0	0	0	1	1	1	
8	0	0	0	1	0	0	0	
9	0	0	0	1	0	0	1	
10	0	0	0	1	0	1	0	

Incrementing a binary counter

value	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	cost
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	1	1
2	0	0	0	0	0	1	0	3
3	0	0	0	0	0	1	1	4
4	0	0	0	0	1	0	0	7
5	0	0	0	0	1	0	1	8
6	0	0	0	0	1	1	0	10
7	0	0	0	0	1	1	1	11
8	0	0	0	1	0	0	0	15
9	0	0	0	1	0	0	1	16
10	0	0	0	1	0	1	0	18

Incrementing a binary counter

A tighter analysis

- Not all bits flip each time INCREMENT is called.
- In general, for $i = 0, 1, \ldots, \lfloor \lg n \rfloor$, bit $A[i]$ flips $\lfloor n/2^i \rfloor$ times in a sequence of n INCREMENT operations on an initially zero counter.
- For $i > \lfloor \lg n \rfloor$, bit $A[i]$ never flips at all.

Incrementing a binary counter

A tighter analysis(cont.)

- The total number of flips in the sequence is thus

$$\sum_{i=0}^{\lfloor \lg n \rfloor} \lfloor n/2^i \rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i}$$

Incrementing a binary counter

A tighter analysis(cont.)

- The total number of flips in the sequence is thus

$$\sum_{i=0}^{\lfloor \lg n \rfloor} \lfloor n/2^i \rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i}$$
$$= 2n.$$

Incrementing a binary counter

A tighter analysis(cont.)

- The worst-case time for a sequence of n INCREMENT operations on an initially zero counter is therefore $O(n)$.
- The average cost of each operaton, and therefore the amortized cost per operation, is $O(n)/n = O(1)$.

Incrementing a binary counter

A tighter analysis(cont.)

- The worst-case time for a sequence of n INCREMENT operations on an initially zero counter is therefore $O(n)$.
- The average cost of each operaton, and therefore the amortized cost per operation, is $O(n)/n = O(1)$.

The accounting method

Definition

- Assign differing charges to different operations, with some operations charged more or less than they actually cost. The amount we charge an operation is called its **amortized cost**. The ith operation's amortized cost is $\hat{c}_i$.
- Any amount not immediately consumed is stored in the bank as **credit** for use by subsequent operations.

The accounting method

Definition

- Assign differing charges to different operations, with some operations charged more or less than they actually cost. The amount we charge an operation is called its **amortized cost**. The ith operation's amortized cost is $\hat{c}_i$.
- Any amount not immediately consumed is stored in the bank as **credit** for use by subsequent operations.

The accounting method

Definition

- The credit in the bank must not go negative! we require

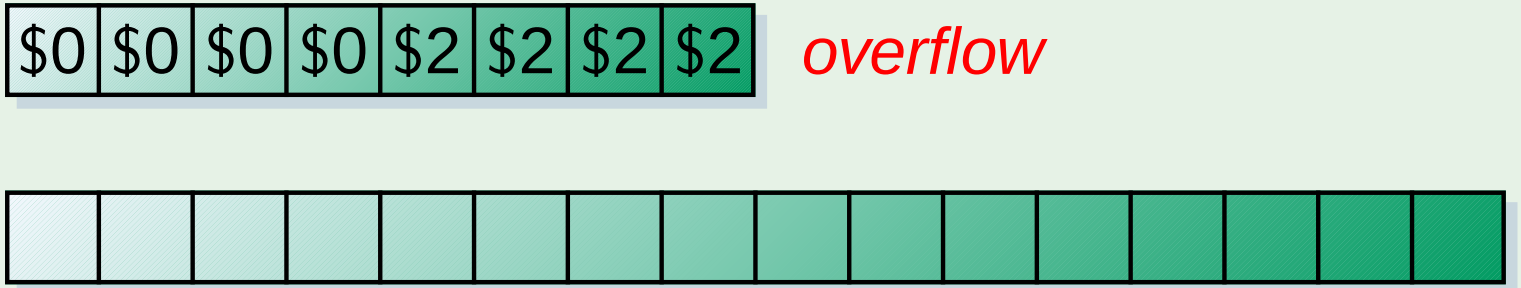
$$\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$$

for all n.

Accounting analysis of dynamic tables

- Charge an amortized cost of $\hat{c}_i = \$3$ for the ith insertion.
  - $\$1$ pays for the immediate insertion.
  - $\$2$ is stored for later table doubling.
- When the table doubles, $\$1$ pays to move a recent item, and $\$1$ pays to move an old item.

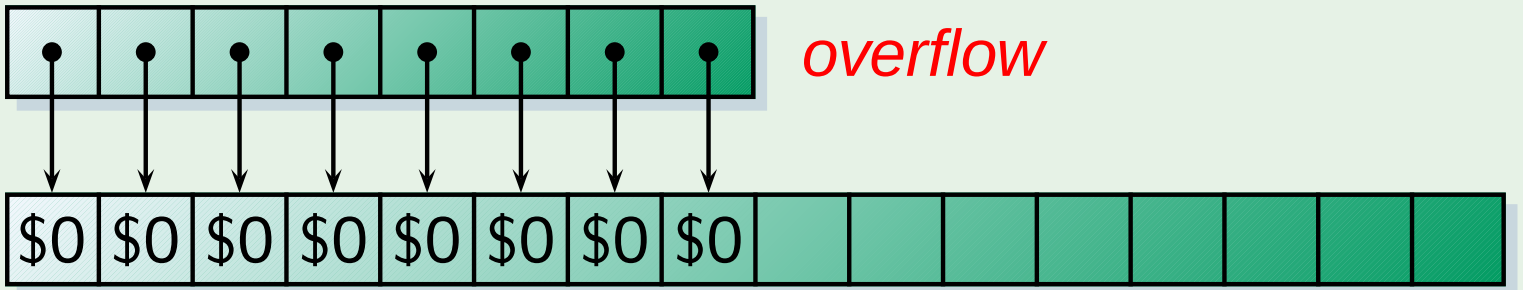
Example



Accounting analysis of dynamic tables

- Charge an amortized cost of $\hat{c}_i = \$3$ for the ith insertion.
  - $\$1$ pays for the immediate insertion.
  - $\$2$ is stored for later table doubling.
- When the table doubles, $\$1$ pays to move a recent item, and $\$1$ pays to move an old item.

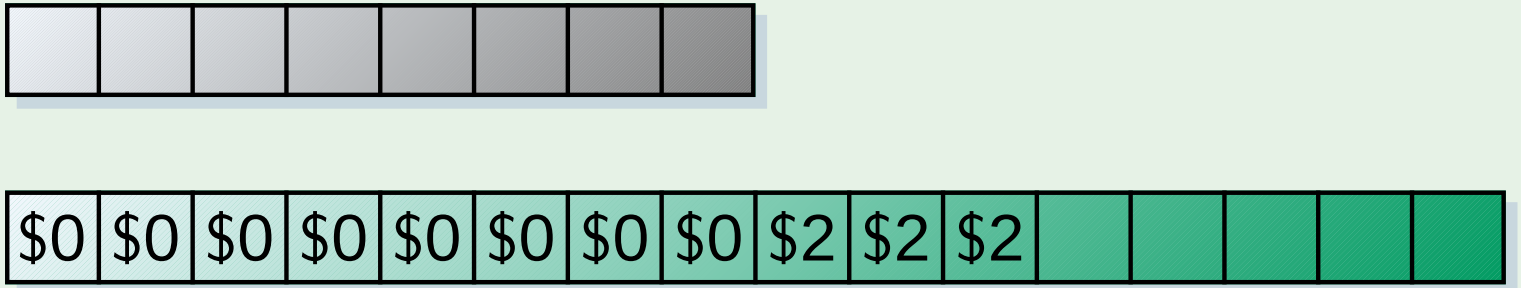
Example



Accounting analysis of dynamic tables

- Charge an amortized cost of $\hat{c}_i = \$3$ for the ith insertion.
  - $\$1$ pays for the immediate insertion.
  - $\$2$ is stored for later table doubling.
- When the table doubles, $\$1$ pays to move a recent item, and $\$1$ pays to move an old item.

Example



Accounting analysis of dynamic tables

Table: Credit never goes negative. Thus, the sum of the amortized costs provides an **upper bound** on the sum of the true costs.

i	1	2	3	4	5	6	7	8	9	10
$size_i$	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
$\hat{c}_i$										
$Credit_i$										

Accounting analysis of dynamic tables

Table: Credit never goes negative. Thus, the sum of the amortized costs provides an **upper bound** on the sum of the true costs.

i	1	2	3	4	5	6	7	8	9	10
$size_i$	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
$\hat{c}_i$	2	3	3	3	3	3	3	3	3	3
$Credit_i$										

Accounting analysis of dynamic tables

Table: Credit never goes negative. Thus, the sum of the amortized costs provides an **upper bound** on the sum of the true costs.

i	1	2	3	4	5	6	7	8	9	10
$size_i$	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
$\hat{c}_i$	2	3	3	3	3	3	3	3	3	3
$Credit_i$	1	2	2	4						

Accounting analysis of dynamic tables

Table: Credit never goes negative. Thus, the sum of the amortized costs provides an **upper bound** on the sum of the true costs.

i	1	2	3	4	5	6	7	8	9	10
$size_i$	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
$\hat{c}_i$	2	3	3	3	3	3	3	3	3	3
$Credit_i$	1	2	2	4	2	4	6	8		

Accounting analysis of dynamic tables

Table: Credit never goes negative. Thus, the sum of the amortized costs provides an **upper bound** on the sum of the true costs.

i	1	2	3	4	5	6	7	8	9	10
$size_i$	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
$\hat{c}_i$	2	3	3	3	3	3	3	3	3	3
$Credit_i$	1	2	2	4	2	4	6	8	2	4

Incrementing a binary counter

- Charge an amortized cost of $\hat{c}_i = \$2$ to set a bit to 1.
  - Use $\$1$ to pay for the actual setting of the bit.
  - Place the other dollar on the bit as credit to be used later when we flip the bit back to 0.
- At any point in time, every 1 in the counter has a dollar of credit on it, and thus we needn't charge anything to reset a bit to 0.

Incrementing a binary counter

- Charge an amortized cost of $\hat{c}_i = \$2$ to set a bit to 1.
  - Use $\$1$ to pay for the actual setting of the bit.
  - Place the other dollar on the bit as credit to be used later when we flip the bit back to 0.
- At any point in time, every 1 in the counter has a dollar of credit on it, and thus we needn't charge anything to reset a bit to 0.

Incrementing a binary counter

- Charge an amortized cost of $\hat{c}_i = \$2$ to set a bit to 1.
  - Use $\$1$ to pay for the actual setting of the bit.
  - Place the other dollar on the bit as credit to be used later when we flip the bit back to 0.
- At any point in time, every 1 in the counter has a dollar of credit on it, and thus we needn't charge anything to reset a bit to 0.

Incrementing a binary counter

- Charge an amortized cost of $\hat{c}_i = \$2$ to set a bit to 1.
  - Use $\$1$ to pay for the actual setting of the bit.
  - Place the other dollar on the bit as credit to be used later when we flip the bit back to 0.
- At any point in time, every 1 in the counter has a dollar of credit on it, and thus we needn't charge anything to reset a bit to 0.

Incrementing a binary counter

- The amortized cost of an INCREMENT operation is at most 2 dollars. ◂ INCREMENT
- The number of 1's in the counter is never negative, and thus the amount of credit is always nonnegative.
- Thus, for n INCREMENT operations, the total amortized cost is $O(n)$.

Incrementing a binary counter

- The amortized cost of an INCREMENT operation is at most 2 dollars. ◂ INCREMENT
- The number of 1's in the counter is never negative, and thus the amount of credit is always nonnegative.
- Thus, for n INCREMENT operations, the total amortized cost is $O(n)$.

Incrementing a binary counter

- The amortized cost of an INCREMENT operation is at most 2 dollars. ◂ INCREMENT
- The number of 1's in the counter is never negative, and thus the amount of credit is always nonnegative.
- Thus, for n INCREMENT operations, the total amortized cost is $O(n)$.

Potential method

Idea

View credit stored as the **potential energy** of the dynamic set.

Framework

1. Start with an initial data structure D_0.
2. Operation i transforms D_{i-1} to D_i.
3. The cost of operation i is c_i.

Potential method

Idea

View credit stored as the **potential energy** of the dynamic set.

Framework

1. Start with an initial data structure D_0.
2. Operation i transforms D_{i-1} to D_i.
3. The cost of operation i is c_i.

Potential method

Idea

View credit stored as the **potential energy** of the dynamic set.

Framework

4. Define a **potential function** $\Phi : \{D_i\} \to \mathbb{R}$, such that $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i.
5. The **amortized cost** $\hat{c}_i$ with respect to Φ is defined to be $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.

Potential method

Understanding potentials

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

- **Potential difference**
  $\Delta\Phi_i = \Phi(D_i) - \Phi(D_{i-1})$.
- If $\Delta\Phi_i > 0$, then $\hat{c}_i > c_i$. Operation i stores work in the data structure for later use.
- If $\Delta\Phi_i < 0$, then $\hat{c}_i < c_i$. The data structure delivers up stored work to help pay for operation i.

Potential method

Understanding potentials

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

- **Potential difference** $\Delta\Phi_i = \Phi(D_i) - \Phi(D_{i-1})$.
- If $\Delta\Phi_i > 0$, then $\hat{c}_i > c_i$. Operation i stores work in the data structure for later use.
- If $\Delta\Phi_i < 0$, then $\hat{c}_i < c_i$. The data structure delivers up stored work to help pay for operation i.

Potential method

Understanding potentials

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

- **Potential difference** $\Delta\Phi_i = \Phi(D_i) - \Phi(D_{i-1})$.
- If $\Delta\Phi_i > 0$, then $\hat{c}_i > c_i$. Operation i stores work in the data structure for later use.
- If $\Delta\Phi_i < 0$, then $\hat{c}_i < c_i$. The data structure delivers up stored work to help pay for operation i.

Potential method

The amortized costs bound the true costs

The total amortized cost of n operations is

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1}))$$
$$= \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0)$$

Potential method

The amortized costs bound the true costs

The total amortized cost of n operations is

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1}))$$
$$= \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0)$$

Potential method

The amortized costs bound the true costs

The total amortized cost of n operations is

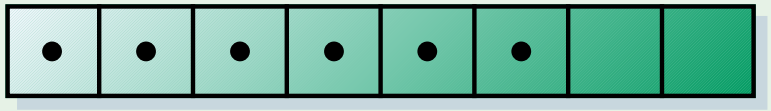
$$\begin{aligned}\sum_{i=1}^{n} \hat{c}_i &= \sum_{i=1}^{n} (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0) \\ &\geq \sum_{i=1}^{n} c_i\end{aligned}$$

Potential analysis of dynamic table

Define the potential of the table after the ith insertion by $\Phi(D_i) = 2 \cdot num_i - size_i$.

- So $\Phi(D_0) = 0$.
- $\Phi(D_i) \geq 0$, for all i.

Example



$$\Phi = 2 \cdot 6 - 8 = 4$$

Potential analysis of dynamic table

Case 1: $i-1$ is not an exact power of 2.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + (2 \cdot num_i - size_i) \\ &\qquad - (2 \cdot num_{i-1} - size_{i-1})\end{aligned}$$

Potential analysis of dynamic table

Case 1: $i-1$ is not an exact power of 2.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + (2 \cdot num_i - size_i) \\ &\qquad - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + (2 \cdot num_i - size_i) \\ &\qquad - (2(num_i - 1) - size_i)\end{aligned}$$

Potential analysis of dynamic table

Case 1: $i-1$ is not an exact power of 2.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + (2 \cdot num_i - size_i) \\ &\qquad - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + (2 \cdot num_i - size_i) \\ &\qquad - (2(num_i - 1) - size_i) \\ &= 3\end{aligned}$$

Potential analysis of dynamic table

Case 2: $i - 1$ is an exact power of 2.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

Potential analysis of dynamic table

Case 2: $i - 1$ is an exact power of 2.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= num_i + (2 \cdot num_i - size_i)) \\ &\qquad - (2 \cdot num_{i-1} - size_{i-1})\end{aligned}$$

Potential analysis of dynamic table

Case 2: $i-1$ is an exact power of 2.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= num_i + (2 \cdot num_i - size_i)) \\ &\qquad - (2 \cdot num_{i-1} - size_{i-1}) \\ &= num_i + (2 \cdot num_i - 2(num_i - 1)) \\ &\qquad - (2(num_i - 1) - (num_i - 1))\end{aligned}$$

Potential analysis of dynamic table

Case 2: $i-1$ is an exact power of 2.

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\
&= num_i + (2 \cdot num_i - size_i)) \\
&\qquad - (2 \cdot num_{i-1} - size_{i-1}) \\
&= num_i + (2 \cdot num_i - 2(num_i - 1)) \\
&\qquad - (2(num_i - 1) - (num_i - 1)) \\
&= num_i + 2 - (num_i - 1)
\end{aligned}
$$

Potential analysis of dynamic table

Case 2: $i-1$ is an exact power of 2.

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= num_i + (2 \cdot num_i - size_i)) \\ &\qquad - (2 \cdot num_{i-1} - size_{i-1}) \\ &= num_i + (2 \cdot num_i - 2(num_i - 1)) \\ &\qquad - (2(num_i - 1) - (num_i - 1)) \\ &= num_i + 2 - (num_i - 1) \\ &= 3\end{aligned}$$

Therefore, n insertions cost $O(n)$ in the worst case.

Table contraction

When to contract?

- When $\alpha(T) = num[T]/size[T] < 1/2$?
- $\alpha(T) < 1/4$ is a good choice.

Potential function

$$\Phi(T) = \begin{cases} 2 \cdot num[T] - size[T] & \text{if } \alpha(T) \geq 1/2, \\ size[T]/2 - num[T] & \text{if } \alpha(T) < 1/2. \end{cases}$$

Table contraction

When to contract?

- When $\alpha(T) = num[T]/size[T] < 1/2$?
- $\alpha(T) < 1/4$ is a good choice.

Potential function

$$\Phi(T) = \begin{cases} 2 \cdot num[T] - size[T] & \text{if } \alpha(T) \geq 1/2, \\ size[T]/2 - num[T] & \text{if } \alpha(T) < 1/2. \end{cases}$$

Table contraction

When to contract?

- When $\alpha(T) = num[T]/size[T] < 1/2$?
- $\alpha(T) < 1/4$ is a good choice.

Potential function

$$\Phi(T) = \begin{cases} 2 \cdot num[T] - size[T] & \text{if } \alpha(T) \geq 1/2, \\ size[T]/2 - num[T] & \text{if } \alpha(T) < 1/2. \end{cases}$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_i < 1/2$

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) \\ &\quad - (size_{i-1}/2 - num_{i-1})\end{aligned}$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_i < 1/2$

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (size_i/2 - num_i) \\
&\qquad - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (size_i/2 - num_i) \\
&\qquad - (size_i/2 - (num_i - 1))
\end{aligned}
$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_i < 1/2$

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (size_i/2 - num_i) \\
&\qquad - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (size_i/2 - num_i) \\
&\qquad - (size_i/2 - (num_i - 1)) \\
&= 0.
\end{aligned}
$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_{i-1} < 1/2$ but $\alpha_i \geq 1/2$

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot num_i - size_i) \\ &\quad - (size_{i-1}/2 - num_{i-1})\end{aligned}$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_{i-1} < 1/2$ but $\alpha_i \geq 1/2$

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (2 \cdot num_i - size_i) \\
&\qquad - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (2(num_{i-1} + 1) - size_{i-1}) \\
&\qquad - (size_{i-1}/2 - num_{i-1})
\end{aligned}
$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_{i-1} < 1/2$ but $\alpha_i \geq 1/2$

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (2 \cdot num_i - size_i) \\
&\qquad - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (2(num_{i-1} + 1) - size_{i-1}) \\
&\qquad - (size_{i-1}/2 - num_{i-1}) \\
&= 3 \cdot num_{i-1} - \frac{3}{2} size_{i-1} + 3
\end{aligned}
$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_{i-1} < 1/2$ but $\alpha_i \geq 1/2$

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 3\alpha_{i-1} size_{i-1} - \frac{3}{2} size_{i-1} + 3\end{aligned}$$

Table contraction

Analysis of TABLE-INSERT when $\alpha_{i-1} < 1/2$ but $\alpha_i \geq 1/2$

$$\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 3\alpha_{i-1} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\
&< \frac{3}{2} size_{i-1} - \frac{3}{2} size_{i-1} + 3 \\
&= 3.
\end{aligned}$$

Table contraction

Analysis of TABLE-DELETE when $\alpha_{i-1} < 1/2, \alpha_i \geq 1/4$

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) \\ &\quad - (size_{i-1}/2 - num_{i-1})\end{aligned}$$

Table contraction

Analysis of TABLE-DELETE when $\alpha_{i-1} < 1/2, \alpha_i \geq 1/4$

$$
\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= 1 + (size_i/2 - num_i) \\
&\qquad - (size_{i-1}/2 - num_{i-1}) \\
&= 1 + (size_i/2 - num_i) \\
&\qquad - (size_i/2 - (num_i + 1)) \\
&= 2.
\end{aligned}
$$

Table contraction

Analysis of TABLE-DELETE when $\alpha_{i-1} < 1/2, \alpha_i < 1/4$

Due $size_i/2 = size_{i-1}/4 = num_{i-1} = num_i + 1$

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= (num_i + 1) + (size_i/2 - num_i) \\ &\qquad - (size_{i-1}/2 - num_{i-1})\end{aligned}$$

Table contraction

Analysis of TABLE-DELETE when $\alpha_{i-1} < 1/2, \alpha_i < 1/4$

Due $size_i/2 = size_{i-1}/4 = num_{i-1} = num_i + 1$

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= (num_i + 1) + (size_i/2 - num_i) \\ &\qquad - (size_{i-1}/2 - num_{i-1}) \\ &= (num_i + 1) + ((num_i + 1) - num_i) \\ &\qquad - ((2 \cdot num_i + 2) - (num_i + 1))\end{aligned}$$

Table contraction

Analysis of TABLE-DELETE when $\alpha_{i-1} < 1/2, \alpha_i < 1/4$

Due $size_i/2 = size_{i-1}/4 = num_{i-1} = num_i + 1$

$$\begin{aligned}
\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\
&= (num_i + 1) + (size_i/2 - num_i) \\
&\qquad - (size_{i-1}/2 - num_{i-1}) \\
&= (num_i + 1) + ((num_i + 1) - num_i) \\
&\qquad - ((2 \cdot num_i + 2) - (num_i + 1)) \\
&= 1.
\end{aligned}$$

Incrementing a binary counter

- We define the ***potential*** of the counter after the ith INCREMENT operation to be b_i, the number of 1's in the counter after the ith operation. See the table

Incrementing a binary counter

- We define the ***potential*** of the counter after the ith INCREMENT operation to be b_i, the number of 1's in the counter after the ith operation. See the table
- The potential difference is

$$\begin{aligned}\Phi(D_i) - \Phi(D_{i-1}) &\leq (b_{i-1} - t_i + 1) - b_{i-1} \\ &= 1 - t_i .\end{aligned}$$

t_i reprents the reseted bits by the ith INCREMENT operation.

Incrementing a binary counter

- The amortized cost is therefore

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &\leq (t_i + 1) + (1 - t_i) \\ &= 2\end{aligned}$$